

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3**



Single Linked List Circular dan Double Linked List Circular

Oleh:

Nuzulia Sekti Mahanania NIM. 2510817220006

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma & Struktur Data Modul 3: Single Linked List Circular dan Double Linked List Circular ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nuzulia Sekti Mahanania
NIM : 2510817220006

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
SOAL 1 : Implementasi Senarai Berantai Tunggal Melingkar	1
A. Source Code.....	1
B. Pembahasan	5
SOAL 2 : Si Palui dan Relikui Siklus Resonansi	8
A. Source Code.....	9
B. Output Program	11
C. Pembahasan	11
SOAL 3 : Implementasi Senarai Berantai Ganda Melingkar	15
A. Source Code.....	15
B. Pembahasan	19
SOAL 4 : Si Palui dan Tarik Tambang Dimensi	22
A. Source Code.....	23
B. Output Program	26
C. Pembahasan	26
TAUTAN GITHUB.....	30

DAFTAR GAMBAR

Gambar 1. Screenshot Output Soal 2 prob_a.cpp.....	11
Gambar 2. Screenshot Output Soal 4 prob_b.cpp	26

DAFTAR TABEL

Tabel 1. Source Code single_linked_list.cpp	1
Tabel 2. Source Code prob_a.cpp.....	9
Tabel 3. Source Code double_linked_list.cpp	15
Tabel 4. Source Code prob_b.cpp	23

SOAL 1 : Implementasi Senarai Berantai Tunggal Melingkar

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari **memory leak** (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan **Big-O Notation**. Seluruh manajemen memori harus dialokasikan secara manual di dalam **heap**.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 1. Source Code single_linked_list.cpp

1	<code>#include <iostream></code>
2	<code>#include "single_linked_list.h"</code>
3	
4	<code>using namespace std;</code>
5	
6	<code>void SingleLinkedList::init() {</code>
7	<code> head = nullptr;</code>
8	<code> tail = nullptr;</code>
9	<code> size = 0;</code>
10	<code>}</code>
11	
12	<code>bool SingleLinkedList::is_empty() {</code>
13	<code> return size == 0;</code>
14	<code>}</code>
15	
16	<code>void SingleLinkedList::add_front(int val) {</code>
17	<code> Node* ptr_new = new Node;</code>
18	<code> ptr_new->data = val;</code>
19	<code> ptr_new->next = nullptr;</code>
20	
21	<code> if (is_empty()) {</code>
22	<code> head = tail = ptr_new;</code>
23	<code> ptr_new->next = head;</code>

```

24     } else {
25         ptr_new->next = head;
26         head = ptr_new;
27         tail->next = head;
28     }
29     size++;
30 }
31
32 void SingleLinkedList::add_back(int val) {
33     Node* ptr_new = new Node;
34     ptr_new->data = val;
35     ptr_new->next = nullptr;
36
37     if (is_empty()) {
38         head = tail = ptr_new;
39         ptr_new->next = head;
40     } else {
41         tail->next = ptr_new;
42         ptr_new->next = head;
43         tail = ptr_new;
44     }
45     size++;
46 }
47
48 void SingleLinkedList::add_idx(int val, int idx) {
49     if (idx < 0 || idx > size) throw "Indeks di luar
50     batas";
51
52     if (idx == 0) { add_front(val); return; }
53     if (idx == size) { add_back(val); return; }
54
55     Node* ptr_new = new Node;
56     ptr_new->data = val;
57     ptr_new->next = nullptr;
58
59     Node* ptr_curr = head;
60     for (int i = 0; i < idx - 1; i++) {
61         ptr_curr = ptr_curr->next;
62     }
63     ptr_new->next = ptr_curr->next;
64     ptr_curr->next = ptr_new;

```

```

64     size++;
65 }
66
67 void SingleLinkedList::delete_front() {
68     if (is_empty()) throw "Senarai kosong";
69
70     if (size == 1) {
71         delete head;
72         head = nullptr;
73         tail = nullptr;
74         size = 0;
75         return;
76     }
77     Node* ptr_del = head;
78     head = head->next;
79     tail->next = head;
80     delete ptr_del;
81     size--;
82 }
83
84 void SingleLinkedList::delete_back() {
85     if (is_empty()) throw "Senarai kosong";
86
87     if (size == 1) {
88         delete_front();
89         return;
90     }
91
92     Node* ptr_curr = head;
93     while (ptr_curr->next != tail) {
94         ptr_curr = ptr_curr->next;
95     }
96     delete tail;
97     tail = ptr_curr;
98     tail->next = head;
99     size--;
100 }
101
102 void SingleLinkedList::delete_idx(int idx) {
103     if (is_empty() || idx < 0 || idx >= size) throw
        "Indeks di luar batas";

```



```

104
105     if (idx == 0) { delete_front(); return; }
106     if (idx == size - 1) { delete_back(); return; }
107
108     Node* ptr_curr = head;
109     for (int i = 0; i < idx - 1; i++) {
110         ptr_curr = ptr_curr->next;
111     }
112     Node* ptr_del = ptr_curr->next;
113     ptr_curr->next = ptr_del->next;
114     delete ptr_del;
115     size--;
116 }
117
118 void SingleLinkedList::display() {
119     if (is_empty()) {
120         cout << "EMPTY" << endl;
121         return;
122     }
123     Node* ptr_curr = head;
124     for (int i = 0; i < size; i++) {
125         cout << ptr_curr->data;
126         if (i < size - 1) {
127             cout << " -> ";
128         }
129         ptr_curr = ptr_curr->next;
130     }
131     cout << endl;
132 }
133
134 int SingleLinkedList::get(int idx) {
135     if (is_empty() || idx < 0 || idx >= size) throw
136         "Indeks di luar batas";
137
138     Node* ptr_curr = head;
139     for (int i = 0; i < idx; i++) {
140         ptr_curr = ptr_curr->next;
141     }
142     return ptr_curr->data;
143 }

```

```

144 void SingleLinkedList::set(int val, int idx) {
145     if (is_empty() || idx < 0 || idx >= size) throw
        "Indeks di luar batas";
146
147     Node* ptr_curr = head;
148     for (int i = 0; i < idx; i++) {
149         ptr_curr = ptr_curr->next;
150     }
151     ptr_curr->data = val;
152 }
153
154 void SingleLinkedList::clear() {
155     while (!is_empty()) {
156         delete_front();
157     }
158 }

```

B. Pembahasan

1. Konsep Dasar dan Alur Algoritma

Senarai Berantai Tunggal Melingkar atau *Circular Single Linked List* adalah struktur penyimpanan data yang berurutan. Pada struktur ini, setiap data dibungkus dalam sebuah wadah bernama *node*. Setiap *node* memiliki sebuah *pointer* yang mengarah ke *node* selanjutnya. Ciri utamanya adalah *node* yang berada di posisi paling akhir (*tail*) selalu memiliki penunjuk yang mengarah kembali ke *node* paling awal (*head*). Hal inilah yang membuat strukturnya menjadi tidak terputus. Pada algoritma ini, ada operasi yang dapat berjalan seketika karena letak datanya sudah diketahui, yaitu fungsi *init*, *is_empty*, *add_front*, *add_back*, dan *delete_front*. Contoh, saat fungsi *add_back* dijalankan untuk menambah data di bagian akhir, program cukup meletakkan *node* baru (*ptr_new*) di belakang *tail* yang lama, lalu memastikan penunjuk dari *node* baru tersebut mengarah kembali ke *head*.

Namun, untuk melakukan proses di tengah barisan seperti pada fungsi *add_idx*, *delete_idx*, *delete_back*, *get*, *set*, dan *display*, program harus menelusuri data satu per satu secara berurutan. Penelusuran ini menggunakan *pointer* bantuan bernama

`ptr_curr` yang berjalan maju dari awal hingga menemukan posisi indeks data yang diminta.

2. Analisis Kompleksitas Waktu dan Ruang (Big-O Notation)

Berdasarkan cara kerjanya, fungsi `init`, `is_empty`, `add_front`, `add_back`, dan `delete_front` memiliki kompleksitas waktu $O(1)$. Ini berarti proses tersebut berjalan secara langsung dalam satu langkah tanpa memedulikan seberapa banyak data yang ada di dalam barisan. Sebaliknya, fungsi yang memerlukan penelusuran seperti `add_idx`, `delete_idx`, `get`, `set`, `display`, serta `delete_back` memiliki kompleksitas waktu $O(N)$. Hal tersebut dikarenakan program harus berjalan membaca barisan dari awal hingga mencapai titik yang dituju, di mana N mewakili jumlah data yang ditelusuri. Untuk kebutuhan kapasitas memori atau kompleksitas ruang, seluruh algoritma ini beroperasi dengan nilai $O(1)$. Artinya, program hanya membutuhkan satu ruang memori tambahan pada satu waktu saat memproses data, tanpa harus menggandakan seluruh struktur data yang sudah ada.

3. Pencegahan Kebocoran Memori

Sesuai larangan penggunaan STL, setiap penambahan data baru harus memesan ruang memori secara manual di area *heap* menggunakan perintah `new Node`. Karena memori di area *heap* bersifat menetap dan tidak hilang otomatis, penumpukan data dapat menyebabkan *memory leak* yang memperlambat sistem. Untuk mencegahnya, fungsi `clear()` dibuat untuk membersihkan sisa memori. Fungsi ini mengulang proses `delete_front()` untuk menghapus *node* satu per satu menggunakan perintah `delete` hingga barisan benar-benar kosong.

4. Validitas dan Kebenaran Algoritma

Algoritma ini dijamin valid karena struktur melingkarnya disusun sesuai *template* yang diberikan dan sudah lolos *auto grader*. Setiap kali ada perubahan data di ujung barisan, penunjuk dari *node* terakhir akan selalu dihubungkan kembali ke bagian kepala senarai. Selain itu, algoritma ini sangat aman karena dilengkapi sistem peringatan menggunakan perintah `throw`. Jika program disuruh memproses indeks yang salah atau menghapus data

saat senarai kosong, program akan menolak dan memberi peringatan agar sistem tidak berhenti mendadak.

SOAL 2 : Si Palui dan Relikui Siklus Resonansi

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K=K+1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K=K-1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi)..

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 10^2$
- $1 \leq K \leq 10^9$
- $1 \leq A_i \leq 10^6$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1.

Format Input

N K

A_1 A_2 ... A_N

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

Penjelasan Contoh Kasus

Pada contoh pertama, susunan batu awal adalah [1,4,3,6,2] dengan $K = 2$. Dari batu pertama, lompatan 2 langkah mendarat di batu 4 sehingga batu tersebut dihancurkan. Karena 4 genap, nilai K bertambah menjadi 3 dan susunan berubah menjadi [1,3,6,2].

Selanjutnya, dengan $K = 3$, lompatan mendarat di batu 2. Batu 2 dihancurkan dan karena genap, K kembali bertambah menjadi 4. Susunan kini menjadi [1,3,6].

Pada fase berikutnya dengan $K = 4$, lompatan mendarat di batu 1. Karena 1 ganjil, setelah dihancurkan nilai K berkurang menjadi 3, menyisakan [3,6].

Terakhir, dengan $K = 3$, lompatan kembali mendarat di batu 3 sehingga batu tersebut dihancurkan. Struktur kini hanya menyisakan batu bernilai 6, sehingga keluaran akhirnya adalah 6.

A. Source Code

Tabel 2. Source Code prob_a.cpp

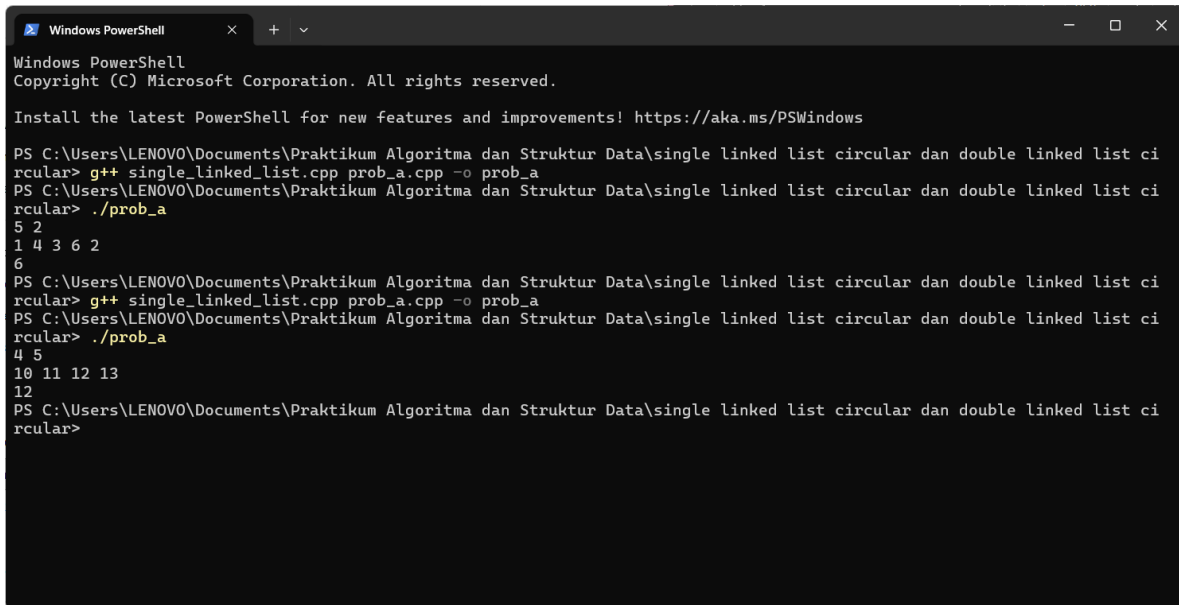
1	#include <iostream>
2	#include "single_linked_list.h"
3	
4	using namespace std;
5	
6	int main() {
7	int N, K;
8	if (!(cin >> N >> K)) return 0;
9	
10	SingleLinkedList list;
11	list.init();
12	
13	for (int i = 0; i < N; i++) {
14	int val;

```

15         cin >> val;
16         list.add_back(val);
17     }
18
19     int init_k = K;
20     int curr_idx = 0;
21
22     while (list.size > 1) {
23         int target_idx = (curr_idx + (K - 1)) % list.size;
24         int target_val = list.get(target_idx);
25
26         list.delete_idx(target_idx);
27
28         curr_idx = target_idx % list.size;
29
30         if (target_val % 2 == 0) {
31             K++;
32         } else {
33             K--;
34         }
35
36         if (K <= 0) {
37             K = init_k;
38         }
39     }
40
41     cout << list.get(0) << endl;
42
43     list.clear();
44     return 0;
45 }

```

B. Output Program



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> g++ single_linked_list.cpp prob_a.cpp -o prob_a
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> ./prob_a
5 2
1 4 3 6 2
6
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> g++ single_linked_list.cpp prob_a.cpp -o prob_a
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> ./prob_a
4 5
10 11 12 13
12
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular>
```

Gambar 1. Screenshot Output Soal 2 prob_a.cpp

C. Pembahasan

1. Penjelasan Baris Kode

Baris [1-2]: Bagian ini digunakan untuk menyertakan *library* yang dibutuhkan oleh program. *Library* *iostream* digunakan untuk proses *input/output* standar, sedangkan "*single_linked_list.h*" disertakan untuk memanggil kerangka struktur data yang telah dibuat sebelumnya.

Baris [4]: Perintah `using namespace std;` digunakan agar penulisan fungsi standar seperti `cin` atau `cout` dapat ditulis secara langsung tanpa perlu diawali dengan instruksi `std::`.

Baris [6]: Merupakan fungsi utama (*main*) di mana seluruh alur eksekusi program pertama kali dimulai.

Baris [7-8]: Mendeklarasikan variabel *N* untuk merepresentasikan jumlah batu awal dan *K* untuk merepresentasikan jarak lompatan energi. Program kemudian membaca *input* dari

pengguna untuk kedua variabel tersebut, dan jika *input* gagal dibaca atau tidak valid, program akan langsung berhenti (`return 0`).

Baris [10-11]: Baris ini membentuk sebuah objek *single linked list* baru dengan nama `list` dari kerangka `SingleLinkedList`. Setelah dibuat, metode `init()` dipanggil untuk memastikan *list* atau senarai dalam kondisi kosong dan siap digunakan.

Baris [13-17]: Merupakan blok perulangan (`for`) yang akan berjalan berulang-ulang sebanyak `N` kali sesuai jumlah batu. Di dalam perulangan ini, program membaca nilai setiap batu (`val`) lalu memasukkannya ke posisi paling belakang pada *list* menggunakan fungsi `add_back(val)`.

Baris [19-20]: Mendeklarasikan variabel `init_k` untuk menyimpan nilai asli dari daya lompatan `K` sebagai cadangan jika nanti terjadi *reset* energi. Selain itu, variabel `curr_idx` disiapkan dengan nilai 0 sebagai titik mula penunjuk lompatan dari batu pertama.

Baris [22]: Memulai proses perulangan utama (`while`) untuk menghancurkan batu. Program diinstruksikan untuk terus berputar dan menghancurkan batu selama sisa jumlah batu di dalam *list* (`list.size`) masih lebih dari 1.

Baris [23-24]: Program menghitung lokasi batu yang akan dihancurkan dengan rumus lompatan dan menyimpannya pada variabel `target_idx`. Penggunaan sisa bagi atau modulo (`% list.size`) berfungsi agar penunjuk posisi berputar kembali ke urutan awal jika lompatannya telah melewati batas ujung senarai. Setelah indeks ditemukan, nilai ukiran pada batu tersebut diambil menggunakan fungsi `get()` dan disimpan di variabel `target_val`.

Baris [26]: Memanggil fungsi `delete_idx()` untuk menghancurkan dan mencabut batu pada posisi *target* dari dalam senarai.

Baris [28]: Memperbarui posisi saat ini (`curr_idx`) agar berpindah ke batu yang tepat berada setelah batu yang baru saja dihancurkan. Modulo kembali pakai sebagai antisipasi agar letak indeks tersebut menyesuaikan ukuran list yang baru menyusut.

Baris [30-34]: Menerapkan aturan anomali kuil. Jika nilai ukiran batu yang dihancurkan adalah genap (`target_val % 2 == 0`), maka kekuatan lompatan K akan bertambah 1. Sebaliknya, jika nilai batu tersebut ganjil, maka daya lompatan K akan berkurang 1.

Baris [36-38]: Menerapkan batasan daya pelindung energi. Jika perhitungan sebelumnya menyebabkan K habis atau bernilai negatif ($K \leq 0$), maka kekuatan lompatan K akan langsung *reset* ke daya semula menggunakan nilai cadangan `init_k`.

Baris [41]: Setelah perulangan `while` selesai dan senarai hanya menyisakan tepat satu buah batu penentu, program akan mengambil dan mencetak nilai batu yang tersisa di posisi pertama `get(0)` tersebut ke layar.

Baris [43-45]: Fungsi `clear()` dipanggil untuk melakukan pembersihan seluruh sisa pesan memori di dalam *heap* agar tidak terjadi *memory leak*. Setelah list kembali bersih tak berbekas, program ditutup dengan perintah `return 0`.

2. Hasil Analisis

Strategi Algoritma:

Masalah utama pada Soal 2 adalah nilai lompatan energi (K) yang sangat besar, yaitu dapat mencapai batas maksimal 10^9 . Apabila program dipaksa untuk berjalan menelusuri list langkah demi langkah sebanyak satu miliar kali pada setiap putarannya, program dipastikan akan melebihi batas waktu eksekusi maksimal 1.0 detik.

Oleh karena itu, strategi penyelesaiannya menggunakan manipulasi matematika sisa bagi (modulo). Indeks batu target dihitung secara langsung menggunakan rumus `(curr_idx + (K - 1)) % list.size`. Rumus modulo ini akan melewati seluruh lompatan yang berputar sia-sia tanpa harus menggeser *pointer* secara manual, dan langsung memberikan posisi indeks pasti dari batu yang harus dihancurkan.

Kompleksitas Waktu $O(N^2)$:

Program menjalankan perulangan `while` yang berputar terus-menerus hingga batu dari total awal N hanya tersisa satu buah. Di dalam setiap perulangan tersebut, program memanggil fungsi pencarian `get()` dan penghapusan `delete_idx()` pada *Single Linked List*. Kedua fungsi ini memerlukan waktu $O(N)$ karena harus menelusuri barisan batu satu per satu dari urutan paling depan. Karena operasi pencarian berwaktu $O(N)$ ini dieksekusi berulang kali di dalam perulangan yang juga berjumlah $O(N)$, maka total akumulasi waktu tempuhnya berlipat ganda menjadi $O(N^2)$.

Kompleksitas Ruang $O(1)$:

Seluruh data ukiran batu sejumlah N yang dimasukkan oleh pengguna di awal program disimpan di dalam satu barisan *Single Linked List*. Karena total pemakaian memori yang dipesan kepada sistem berbanding lurus dan tumbuh secara linear mengikuti banyaknya input batu N , maka kompleksitas ruangnya adalah $O(N)$.

SOAL 3 : Implementasi Senarai Berantai Ganda Melingkar

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList`. beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari **memory leak** (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan **Big-O Notation**. Seluruh manajemen memori harus dialokasikan secara manual di dalam **heap**.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 3. Source Code double_linked_list.cpp

1	<code>#include <iostream></code>
2	<code>#include "double_linked_list.h"</code>
3	
4	<code>using namespace std;</code>
5	
6	<code>void DoubleLinkedList::init() {</code>
7	<code> head = nullptr;</code>
8	<code> tail = nullptr;</code>
9	<code> size = 0;</code>
10	<code>}</code>
11	
12	<code>bool DoubleLinkedList::is_empty() {</code>
13	<code> return size == 0;</code>
14	<code>}</code>
15	
16	<code>void DoubleLinkedList::add_front(char val) {</code>
17	<code> Node* ptr_new = new Node;</code>
18	<code> ptr_new->data = val;</code>
19	<code> ptr_new->next = nullptr;</code>
20	<code> ptr_new->prev = nullptr;</code>
21	
22	<code> if (is_empty()) {</code>
23	<code> head = ptr_new;</code>
24	<code> tail = ptr_new;</code>

```

25         ptr_new->next = ptr_new;
26         ptr_new->prev = ptr_new;
27     } else {
28         ptr_new->next = head;
29         ptr_new->prev = tail;
30         head->prev = ptr_new;
31         tail->next = ptr_new;
32         head = ptr_new;
33     }
34     size++;
35 }
36
37 void DoubleLinkedList::add_back(char val) {
38     Node* ptr_new = new Node;
39     ptr_new->data = val;
40     ptr_new->next = nullptr;
41     ptr_new->prev = nullptr;
42
43     if (is_empty()) {
44         head = ptr_new;
45         tail = ptr_new;
46         ptr_new->next = ptr_new;
47         ptr_new->prev = ptr_new;
48     } else {
49         ptr_new->prev = tail;
50         ptr_new->next = head;
51         tail->next = ptr_new;
52         head->prev = ptr_new;
53         tail = ptr_new;
54     }
55     size++;
56 }
57
58 void DoubleLinkedList::add_idx(char val, int idx) {
59     if (idx < 0 || idx > size) return;
60
61     if (idx == 0) { add_front(val); return; }
62     if (idx == size) { add_back(val); return; }
63
64     Node* ptr_curr = head;
65     for (int i = 0; i < idx; i++) {

```

```

66         ptr_curr = ptr_curr->next;
67     }
68
69     Node* ptr_new = new Node;
70     ptr_new->data = val;
71     ptr_new->next = ptr_curr;
72     ptr_new->prev = ptr_curr->prev;
73
74     ptr_curr->prev->next = ptr_new;
75     ptr_curr->prev = ptr_new;
76     size++;
77 }
78
79 void DoubleLinkedList::delete_front() {
80     if (is_empty()) return;
81     if (size == 1) {
82         delete head;
83         head = nullptr;
84         tail = nullptr;
85         size = 0;
86         return;
87     }
88     Node* ptr_del = head;
89     head = head->next;
90     head->prev = tail;
91     tail->next = head;
92     delete ptr_del;
93     size--;
94 }
95
96 void DoubleLinkedList::delete_back() {
97     if (is_empty()) return;
98     if (size == 1) {
99         delete_front();
100         return;
101     }
102
103     Node* ptr_del = tail;
104     tail = tail->prev;
105     tail->next = head;
106     head->prev = tail;

```

```

107     delete ptr_del;
108     size--;
109 }
110
111 void DoubleLinkedList::delete_idx(int idx) {
112     if (is_empty() || idx < 0 || idx >= size) return;
113
114     if (idx == 0) { delete_front(); return; }
115     if (idx == size - 1) { delete_back(); return; }
116
117     Node* ptr_curr = head;
118     for (int i = 0; i < idx; i++) {
119         ptr_curr = ptr_curr->next;
120     }
121     ptr_curr->prev->next = ptr_curr->next;
122     ptr_curr->next->prev = ptr_curr->prev;
123     delete ptr_curr;
124     size--;
125 }
126
127 void DoubleLinkedList::display() {
128     if (is_empty()) {
129         cout << "EMPTY" << endl;
130         return;
131     }
132     Node* ptr_curr = head;
133     for (int i = 0; i < size; i++) {
134         cout << ptr_curr->data;
135         if (i < size - 1) {
136             cout << " <-> ";
137         }
138         ptr_curr = ptr_curr->next;
139     }
140     cout << endl;
141 }
142
143 char DoubleLinkedList::get(int idx) {
144     if (is_empty() || idx < 0 || idx >= size) return
145     '\0';
146     Node* ptr_curr = head;
147     for (int i = 0; i < idx; i++) {

```

```

147         ptr_curr = ptr_curr->next;
148     }
149     return ptr_curr->data;
150 }
151
152 void DoubleLinkedList::set(char val, int idx) {
153     if (is_empty() || idx < 0 || idx >= size) return;
154     Node* ptr_curr = head;
155     for (int i = 0; i < idx; i++) {
156         ptr_curr = ptr_curr->next;
157     }
158     ptr_curr->data = val;
159 }
160
161 void DoubleLinkedList::clear() {
162     while (!is_empty()) {
163         delete_front();
164     }
165 }

```

B. Pembahasan

1. Konsep Dasar dan Alur Algoritma

Senarai Berantai Ganda Melingkar atau *Circular Double Linked List* adalah struktur data di mana setiap *node* memiliki dua buah penunjuk arah, yaitu penunjuk *next* untuk menyambung ke *node* setelahnya dan penunjuk *prev* untuk menyambung ke *node* sebelumnya. Sifat melingkarnya terbentuk karena penunjuk *next* pada *node* paling akhir (*tail*) mengarah kembali ke *node* paling awal (*head*), dan sebaliknya penunjuk *prev* pada *head* mengarah kembali ke *tail*. Pada algoritma ini, operasi penambahan dan penghapusan di bagian ujung seperti *init*, *is_empty*, *add_front*, *add_back*, *delete_front*, dan *delete_back* dapat langsung dieksekusi tanpa pencarian.

Keunggulan senarai ganda terlihat pada fungsi *delete_back* (menghapus data terakhir), program tidak perlu lagi menelusuri data dari awal karena *node* sebelum *tail* dapat langsung diketahui melalui penunjuk *prev*. Sementara itu, untuk proses yang melibatkan indeks spesifik di tengah barisan seperti *add_idx*, *delete_idx*, *get*, *set*, dan *display*, program tetap harus menelusuri data secara berurutan. Penelusuran ini

dilakukan dengan menggunakan *pointer* bantuan `ptr_curr` yang bergerak langkah demi langkah dari kepala senarai hingga mencapai letak posisi yang diminta.

2. Analisis Kompleksitas Waktu dan Ruang (Big-O Notation)

Berdasarkan alur kerjanya, kompleksitas waktu program ini terbagi menjadi dua. Fungsi operasi ujung seperti `init`, `is_empty`, `add_front`, `add_back`, `delete_front`, dan `delete_back` memiliki kompleksitas waktu $O(1)$. Hal ini berarti prosesnya selesai dalam satu instruksi dasar, tanpa terpengaruh oleh seberapa panjang barisan data yang ada. Sebaliknya, fungsi yang mewajibkan penelusuran indeks seperti `add_idx`, `delete_idx`, `get`, `set`, dan `display` beroperasi pada kompleksitas waktu $O(N)$. Nilai N pada notasi tersebut mewakili banyaknya data yang harus dilalui oleh program untuk menemukan letak indeks yang ditargetkan.

Dari sisi kapasitas, seluruh fungsi di dalam algoritma ini memiliki kompleksitas ruang sebesar $O(1)$. Hal ini dikarenakan program hanya membuat penunjuk memori sementara yang sangat kecil secara bergiliran saat proses berlangsung, tanpa pernah menduplikasi seluruh susunan data menjadi bentuk yang baru.

3. Pencegahan Kebocoran Memori

Sama seperti *Circular Single Linked List* dan sesuai larangan penggunaan STL, setiap penambahan data baru harus memesan ruang memori secara manual di area *heap* menggunakan perintah `new Node`. Karena memori di area *heap* bersifat menetap dan tidak hilang secara otomatis, penumpukan data lama dapat menyebabkan *memory leak* yang memberatkan komputer. Untuk mencegahnya, fungsi `clear()` diimplementasikan untuk membersihkan memori. Fungsi ini bekerja dengan mengulang proses `delete_front()` guna menghapus *node* satu per satu melalui perintah `delete` hingga barisan kembali kosong sempurna.

4. Validitas dan Kebenaran Algoritma

Algoritma ini dijamin valid karena fungsi yang digunakan sudah sesuai *template* yang diberikan dan berhasil diimplementasikan ke dalam kasus *prob_b*, meskipun tidak lolos *auto grader*. Pada setiap perubahan letak data di ujung barisan, penunjuk silang `tail->next` dan `head->prev` selalu dihubungkan kembali secara bersamaan untuk mencegah

putusnya rantai. Algoritma ini juga memiliki keamanan yang apabila program menerima instruksi di luar nalar (seperti memproses indeks minus), program akan langsung menolak dan menghentikan proses menggunakan perintah `return` sehingga struktur memori terhindar dari kerusakan sistem.

SOAL 4 : Si Palui dan Tarik Tambang Dimensi

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari `head`, bergerak maju menggunakan `next`) dan Beta (dimulai dari `tail`, bergerak mundur menggunakan `prev`).

Palui melakukan observasi selama R ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter **next** dari karakter yang hancur, dan Beta berpindah ke karakter **prev** dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi.

Batasan

- $1 \leq N \leq 5000$
- $1 \leq R \leq 5000$
- $1 \leq X, Y \leq 10^{18}$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Format Input

N R

C_1 C_2 ... C_n

X_1 Y_1

X_2 Y_2

...

XR YR

Keterangan: Ci adalah satu karakter `char` tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY.

Contoh Kasus

Input	Output
4 2 A B C D 1 1 2 3	ACB
5 3 V W X Y Z 100000 100000 2 2 5 5	VWYZ

Penjelasan Contoh Kasus

Pada contoh pertama, awalnya proyektor Alpha berada di node A dan proyektor B berada di node B. Pada ronde pertama Alpha maju satu langkah dan Beta mundur satu langkah, sehingga Alpha berada di node B dan Beta berada di node C. Karena bersebelahan, maka node B dan C di-swap, menjadi A C B D. Kemudian di ronde kedua, Alpha bergerak maju 2 langkah dan Beta mundur 3 langkah, sehingga keduanya berada di node D. Karena keduanya di node yang sama, maka node D dihapus. Sehingga hasil akhirnya ACB.

A. Source Code

Tabel 4. Source Code `prob_b.cpp`

1	<code>#include <iostream></code>
2	<code>#include "double_linked_list.h"</code>
3	
4	<code>using namespace std;</code>
5	
6	<code>int main() {</code>
7	<code> int N, R;</code>
8	<code> if (!(cin >> N >> R)) return 0;</code>

```

9
10     DoubleLinkedList list;
11     list.init();
12
13     for (int i = 0; i < N; i++) {
14         char input_char;
15         cin >> input_char;
16         list.add_back(input_char);
17     }
18
19     Node* ptr_alpha = list.head;
20     Node* ptr_beta = list.tail;
21
22     for (int i = 0; i < R; i++) {
23         long long X, Y;
24         if (!(cin >> X >> Y)) break;
25
26         if (list.size == 0) {
27             break;
28         }
29
30         long long step_alpha = X % list.size;
31         for (long long j = 0; j < step_alpha; j++) {
32             ptr_alpha = ptr_alpha->next;
33         }
34
35         long long step_beta = Y % list.size;
36         for (long long j = 0; j < step_beta; j++) {
37             ptr_beta = ptr_beta->prev;
38         }
39
40         if (ptr_alpha == ptr_beta) {
41             Node* ptr_del = ptr_alpha;
42
43             if (list.size == 1) {
44                 delete ptr_del;
45                 list.head = nullptr;
46                 list.tail = nullptr;
47                 list.size = 0;
48                 ptr_alpha = nullptr;
49                 ptr_beta = nullptr;

```

```

50         } else {
51             ptr_alpha = ptr_del->next;
52             ptr_beta = ptr_del->prev;
53
54             ptr_del->prev->next = ptr_del->next;
55             ptr_del->next->prev = ptr_del->prev;
56
57             if (list.head == ptr_del) {
58                 list.head = ptr_del->next;
59             }
60             if (list.tail == ptr_del) {
61                 list.tail = ptr_del->prev;
62             }
63
64             delete ptr_del;
65             list.size--;
66         }
67     }
68     else if (ptr_alpha->next == ptr_beta ||
69 ptr_alpha->prev == ptr_beta) {
70         bool is_edge = (ptr_alpha == list.head &&
71 ptr_beta == list.tail) || (ptr_alpha == list.tail &&
72 ptr_beta == list.head);
73         if (!(is_edge && list.size > 2)) {
74             char temp = ptr_alpha->data;
75             ptr_alpha->data = ptr_beta->data;
76             ptr_beta->data = temp;
77         }
78     }
79     if (list.size == 0) {
80         cout << "EMPTY" << endl;
81     } else {
82         Node* ptr_curr = list.head;
83         for (int i = 0; i < list.size; i++) {
84             cout << ptr_curr->data;
85             ptr_curr = ptr_curr->next;
86         }
87         cout << endl;
88     }

```

88	
89	<code>list.clear();</code>
90	<code>return 0;</code>
91	<code>}</code>

B. Output Program

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> g++ double_linked_list.cpp prob_b.cpp -o prob_b
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> ./prob_b
4 2
A B C D
1 1
2 3
ACB
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> g++ double_linked_list.cpp prob_b.cpp -o prob_b
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> ./prob_b
5 3
V W X Y Z
100000 100000
2 2
5 5
VWYZ
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular>

```

Gambar 2. Screenshot Output Soal 4 prob_b.cpp

C. Pembahasan

1. Penjelasan Baris Kode

Baris [1-2]: Bagian ini bertugas untuk mengimpor *library* bagi berjalannya program. *Library* <iostream> disertakan untuk masukan dan keluaran standar, sedangkan berkas *header* "double_linked_list.h" diimpor agar program utama dapat mengenali dan menggunakan struktur data *Circular Double Linked List*.

Baris [4-6]: Perintah `using namespace std;` dideklarasikan agar penulisan kode menjadi efisien, sehingga pemanggilan fungsi tidak perlu diawali dengan `std::`. Selanjutnya, baris 6 mendefinisikan `int main()`, fungsi utama dan titik awal sistem operasi mulai mengeksekusi logika program.

Baris [7-11]: Program mendeklarasikan variabel `N` untuk merepresentasikan jumlah karakter awal dan `R` untuk menentukan seberapa banyak ronde observasi yang akan dilakukan. Setelah program membaca *input* untuk kedua variabel tersebut, sebuah objek *list* baru bernama `list` diciptakan. Fungsi `init()` kemudian langsung dipanggil untuk mengamankan *list* agar benar-benar kosong dan bebas dari data sisa di dalam memori.

Baris [13-17]: Program menjalankan perulangan `for` sebanyak `N` kali untuk menyerap karakter. Pada setiap putarannya, sebuah karakter diserap ke dalam variabel `input_char` dan langsung disisipkan ke posisi paling akhir *list* menggunakan fungsi `add_back()` untuk menjamin urutannya sesuai dengan *input*.

Baris [19-20]: Fase peletakan posisi awal bagi kedua proyektor observasi. *Pointer* `ptr_alpha` diletakkan di titik awal barisan dengan merujuk pada `list.head`, sementara *pointer* `ptr_beta` diletakkan di titik akhir barisan dengan merujuk pada `list.tail`.

Baris [22-28]: Program memulai blok perulangan utama yang akan mengeksekusi proses lompatan sebanyak `R` ronde. Pada awal setiap ronde, program membaca besaran lompatan `X` (untuk Alpha) dan `Y` (untuk Beta). Baris 26-28 berfungsi sebagai sistem keamanan, apabila *list* sudah terdeteksi kosong sepenuhnya (`list.size == 0`), program akan langsung menghentikan sisa ronde menggunakan perintah `break` untuk mencegah *error* pembacaan memori.

Baris [30-38]: Bagian inti pengoptimalan waktu eksekusi. Mengingat nilai lompatan bisa bernilai sangat besar, program menggunakan operasi sisa bagi (modulo) terhadap ukuran *list* (`X % list.size` dan `Y % list.size`) untuk mendapatkan jumlah langkah efektif dan melewati putaran yang sia-sia. Setelah didapatkan jumlah langkah efektifnya, `ptr_alpha` digerakkan maju menggunakan *pointer* `next`, dan secara bersamaan `ptr_beta` digerakkan mundur menggunakan *pointer* `prev`.

Baris [40-67]: Blok kode ini menangani kasus "Tabrakan Singular", yaitu saat kedua proyektor mendarat di *node* yang sama persis (`ptr_alpha == ptr_beta`). Memori

target disimpan ke dalam variabel `ptr_del`. Baris (43-49), jika ukuran *list* hanya tersisa satu, *node* tersebut langsung dihancurkan dan ukuran *list* dikosongkan total. Baris (50-66), jika elemen masih lebih dari satu, posisi proyektor diselamatkan terlebih dahulu (Alpha maju, Beta mundur). Setelah itu, *pointer node* sebelum dan sesudahnya disambung ulang agar lingkaran *list* tidak terputus. Program juga memperbarui acuan `list.head` dan `list.tail` jika perlu, lalu menghapus *node* (`delete ptr_del`) serta mengurangi ukuran *list*.

Baris [68-75]: Blok ini difokuskan untuk menangani "Resonansi Bersebelahan" melalui fungsi `else if`. Algoritma mendeteksi apakah posisi Alpha dan Beta saling menempel secara berdampingan. Jika terbukti berdampingan, program akan menukar data ukiran yang dibawa oleh kedua *node* tersebut. Proses pertukaran ini menggunakan variabel bantuan `temp` untuk memegang data sementara, sehingga pertukaran nilai antara Alpha dan Beta berjalan lancar tanpa menghilangkan salah satu data.

Baris [78-87]: Pencetakan hasil akhir. Program mengevaluasi *list*, jika hasil menunjukkan `list.size == 0`, program mencetak teks "EMPTY". Namun, jika data masih ada, program menggunakan *pointer* bantuan `ptr_curr` yang dimulai dari kepala *list* untuk menelusuri dan mencetak karakter demi karakter ke layar hingga mencapai akhir barisan.

Baris [89-91]: Sebagai langkah standar pembersihan pada C++, fungsi `clear()` dipanggil untuk melakukan perulangan penghapusan sisa-sisa *node* di dalam ruang *heap*. Pembersihan ini mutlak dilakukan untuk menjamin sistem terhindar dari kebocoran memori. Setelah pembersihan selesai, program ditutup dengan instruksi `return 0` pada baris 90.

2. Hasil Analisis

Strategi Algoritma :

Tantangan utama pada algoritma ini adalah besaran nilai lompatan proyektor Alpha (X) dan Beta (Y) yang sangat besar, yakni dapat mencapai batas 10^{18} . Jika program dipaksa menelusuri memori satu per satu sebanyak nilai tersebut, program dipastikan akan

memakan waktu terlalu lama dan gagal . Oleh karena itu, strategi optimasi yang digunakan adalah memanfaatkan sifat melingkar dari struktur data ini melalui operasi sisa bagi atau modulo. Dengan menerapkan rumus $X \% list.size$ dan $Y \% list.size$, program membuang seluruh putaran penuh yang berulang secara sia-sia dan langsung menghasilkan langkah efektifnya. Dengan ini, sejauh apa pun instruksi yang diberikan, *pointer* proyektor hanya perlu bergeser maksimal sebanyak jumlah elemen yang ada di dalam list, sehingga dapat berjalan dengan sangat cepat.

Kompleksitas Waktu $O(R \times N)$:

Program melakukan perulangan utama sebanyak R kali yang mewakili jumlah ronde observasi. Di dalam setiap ronde itu, proyektor Alpha dan Beta berjalan menelusuri list untuk mencari lokasi baru. Karena langkah ini telah dibatasi oleh rumus modulo, penelusuran maksimal yang dilakukan oleh proyektor adalah sepanjang jumlah data yang tersisa, yang memakan waktu $O(N)$. Setelah letak memori ditemukan, operasi pada *Double Linked List* seperti pencabutan *node* atau pertukaran nilai dapat dieksekusi seketika dalam waktu $O(1)$. Karena penelusuran berwaktu $O(N)$ ini dijalankan berulang kali di dalam perulangan sebanyak R ronde, maka total kompleksitas waktunya berlipat ganda menjadi $O(R \times N)$.

Kompleksitas Ruang $O(N)$:

Program menyimpan rentetan karakter input dari pengguna ke dalam memori penyusun struktur *Circular Double Linked List*. Pemesanan ruang memori di area *heap* ini dialokasikan secara presisi mengikuti jumlah input awal N . Selama ronde observasi berlangsung, penggunaan ruang memori ini tidak pernah bertambah melewati ukuran aslinya, dan bahkan akan menyusut saat ada karakter yang dihapus akibat tabrakan. Karena kapasitas memori yang dibutuhkan murni berbanding lurus dan tumbuh secara linear mengikuti besaran input awal N , maka kompleksitas ruangnya dikategorikan sebagai $O(N)$.

TAUTAN GITHUB

<https://github.com/DSA26-ULM/module-3-linked-list-nullrym>